

Design and Evaluation of a Distributed Hybrid System for Low-Latency Semantic and Structured Information Retrieval

Msc Computer Science



**UNIVERSITY
OF LONDON**

Software Project Report
Devang Chauhan

XXXX Words

Excluding title page, abstract, table of content, tables, references and appendix

Abstract

Placeholder

Table of Contents

Abstract	2
Table of Contents	3
List of Figures	4
List of Tables.....	4
1. Introduction.....	5
2. Literature Review	6
2.1 RAG.....	6
2.2 NL2SQL.....	6
2.3 Resource Selection	7
2.4 Vector Search	7
2.5 Relational Search	7
2.5 Distributed Systems	8
2.6 Query Planning.....	8
2.7 Contribution	8
3. Methodology	10
4. Results and Analysis	11
5. Evaluation.....	12
6. Conclusion.....	13
References	14
Appendices.....	17

List of Figures

Figure - 1 : Sample Placeholder	4
---------------------------------------	---

List of Tables

Table - 2: Sample Placeholder	4
-------------------------------------	---

Figure - 1 : Sample Placeholder

Table - 2: Sample Placeholder

1. Introduction

The advent of large language models (LLMs) [1] have opened up numerous applications where a conversation can be had with an LLM using the knowledge encoded in the parameters of the model during pre-training. While fascinating, this has a limitation when it comes to their use within enterprises. Enterprise data is often gated and hence is not included in the pre-training data of the LLMs. This can lead to hallucinations where the LLM confidently responds to queries with false information where it does not have innate knowledge. Retrieval augmented generation (RAG) [2], while based on techniques long known in the domains of search and recommendations systems, has been found to be a reliable way to ground the responses of LLMs in relevant and recent information to reduce hallucinations [3].

RAG is extensively used to provide gated enterprise knowledge to LLMs to ground their responses. Depending on the shape of information the enterprise knowledge is often distributed across several knowledge stores such as vector stores, document stores, relational tables and graph databases. As this knowledge grows, the knowledge retrieval systems start taking the form of a distributed system where these knowledge stores needs to be either partitioned or replicated or both depending on whether consistency or availability is targeted [4].

In any such information retrieval system, it is important to maintain a balance between the speed and accuracy of the response served to the user to maintain a higher degree of user delight [5]. Therefore, the aim of this project is to design an orchestrator and evaluate how different orchestration and routing strategies affect latency, and retrieval quality in distributed heterogeneous retrieval systems.

In the information-retrieval literature, hybrid retrieval [6] commonly refers to combining sparse lexical and dense semantic retrieval signals into a unified ranking. The system developed in this project does not perform this form of score or rank fusion. Instead, it is a heterogeneous retrieval system (HRS) supporting structured relational, dense semantic and sparse lexical retrieval over aligned distributed data using a unified orchestration.

This report begins by reviewing the work in this area. It then moves on to explain the methodology adopted in design of the orchestrator, followed by the results recorded in the use of the orchestrator against a gold standard benchmark, and lastly, the evaluation and discussion of those results to comment on their speed and accuracy.

2. Literature Review

This project lies at the intersection of RAG, natural language to SQL (NL2SQL) [7], federated search, vector search, relational databases and distributed systems. Consequently, the literature review draws upon contributions from each of these domains to motivate the proposed architecture and identify the research gap.

2.1 RAG

RAG has received considerable attention from the research community. Xu et al. [8] propose a distributed RAG (DRAG) system using a peer-to-peer (P2P) network of knowledge stores and local LLMs. Their healthcare-focused approach introduces a novel algorithm that retrieves information locally before querying neighbouring peers. While the system is distributed, it relies solely on vector embeddings as grounding information for LLMs and therefore does not address an HRS, where structured data must also be queried. Similarly, Guerraoui et al. [9] acknowledge the distributed nature of modern retrieval systems and propose RAGROUTE, a neural network classifier that predicts which knowledge store should be queried. Although this reduces unnecessary communication, the routing model is probabilistic and requires retraining whenever the underlying knowledge stores are updated. Recognising that enterprise retrieval extends beyond embeddings, Yu et al. [10] propose TableRAG, which embeds relational table schemas alongside textual content so that retrieved schema embeddings can guide subsequent SQL queries. While this demonstrates the value of heterogeneous retrieval, the paper does not discuss schema evolution and how very large distributed tables are handled in practice.

2.2 NL2SQL

As structured data becomes an increasingly important component of HRS, the quality of NL2SQL translation has a direct impact on retrieval quality. Existing approaches can broadly be categorised as rule-based, neural and hybrid methods. Shalaan et al. [11] propose Guided SQL (GSQL), which combines rule-based processing, natural language processing and vector embeddings to improve SQL generation. SQLizer adopts a different approach by first constructing a query sketch before iteratively materialising it against the database schema using CoreNLP and Word2Vec [12]. More recently, Liu et al. [7] observe that although rule-based systems provide explainability, they struggle with adaptability and complex schemas, while neural approaches remain constrained by training data. Consequently, the field has shifted towards LLM-based methods and fine-tuning techniques that demonstrate stronger generalisation across complex database schemas [7]. Given this direction, implementing a custom NL2SQL engine would contribute relatively little to the

primary aim of this project, which is the design and evaluation of query orchestration rather than query translation itself.

2.3 Resource Selection

Once queries have been translated, the retrieval system must determine which resources should be searched. Federated search addresses this problem by providing a unified interface through which multiple independent collections can be searched and their results combined [13]. Garba et al. [13] identify CORI [14] and ReDDE [15] as two of the most influential resource selection techniques. Callan et al. [14] use probabilistic inference networks to estimate the likelihood that a collection contains relevant information. Here, CORI relies primarily on term frequency (TF) and inverse document frequency (IDF), limiting its ability to capture semantic similarity. Si et al. [15] improve resource selection by estimating collection sizes and emphasise that reducing communication between distributed collections becomes increasingly important as the number of knowledge stores grows. These studies establish that resource selection remains a fundamental challenge in distributed retrieval, although most existing approaches were developed for keyword-based search rather than modern heterogeneous retrieval systems.

2.4 Vector Search

Semantic retrieval addresses the resource selection by representing documents and queries as vector embeddings, allowing similarity to be measured using distance functions such as cosine similarity or Euclidean distance rather than exact keyword matching [4]. However, efficiently searching large vector collections presents another challenge. Flat indexes provide exact retrieval but exhibit linear search complexity, making them unsuitable for large datasets [4]. Modern vector databases therefore employ approximate nearest neighbour (ANN) indexes such as inverted file index (IVF) and hierarchical navigable small world index (HNSW), which significantly reduce retrieval latency while maintaining high retrieval quality [4, 16]. More recent approaches, including DiskANN and SPANN, further improve scalability by storing large portions of the index on SSDs while maintaining lightweight routing structures in memory, enabling efficient retrieval over billion-scale datasets [17, 18]. These approaches substantially improve vector search performance but focus on optimising retrieval within individual vector stores rather than across heterogeneous and distributed knowledge stores.

2.5 Relational Search

While semantic retrieval addresses unstructured information, enterprise knowledge also resides within relational databases. Relational systems typically employ B-tree

indexes for range queries, hash indexes for equality comparisons and GIN indexes for full-text and JSON search [4, 19]. Kleppmann and Riccomini [4] note that although indexes improve retrieval performance, they increase storage requirements and slow write operations because they must be maintained as the underlying data changes. Consequently, index selection represents a trade-off between query performance and maintenance overhead.

2.5 Distributed Systems

As enterprise knowledge continues to grow, a single machine may no longer be sufficient to store or process all data. Distributed database systems therefore introduce concerns such as sharding and replication. The degree of fragmentation directly influences query performance [20], while the choice of sharding strategy depends heavily on workload characteristics. Key-range partitioning is effective when queries exhibit locality, whereas hash-based partitioning generally provides a more balanced distribution of reads and writes [4]. Similar principles apply to vector databases, where semantically similar vectors may be clustered together to reduce the search space during retrieval [21]. Distributed systems also introduce opportunities for parallel query execution; however, Özsu and Valduriez [22] note that reducing response time through parallelism may increase overall execution time because of additional communication overhead. Consequently, computation should occur as close to the data as possible whenever feasible [22].

2.6 Query Planning

These considerations ultimately influence query planning within distributed retrieval systems. Özsu and Valduriez [23] argue that query optimisation should minimise communication costs by reducing the amount of data transferred between nodes. They recommend assigning execution costs to alternative query plans and favour strategies that perform selection as early as possible, thereby reducing intermediate results and avoiding expensive join operations wherever possible [23]. Such observations reinforce the importance of orchestration strategies that intelligently coordinate retrieval across multiple heterogeneous knowledge stores.

2.7 Contribution

Collectively, the literature demonstrates significant progress in semantic retrieval, distributed RAG systems, vector indexing and hybrid retrieval. Existing work has largely focused on improving individual components of the retrieval pipeline, including vector search, NL2SQL translation and resource selection. Comparatively little research, however, evaluates how deterministic orchestration strategies influence latency and retrieval effectiveness in distributed hybrid retrieval systems that combine

semantic vector search with structured relational querying. This project addresses that gap by designing and experimentally evaluating alternative routing and orchestration strategies under a sharded knowledge store using established information retrieval and distributed systems performance metrics.

3. Methodology

Placeholder

4. Results and Analysis

Placeholder

5. Evaluation

Placeholder

6. Conclusion

Placeholder

References

- [1] T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 1877–1901. Accessed: Sep. 17, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [2] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2020, pp. 9459–9474. Accessed: Sep. 17, 2026. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html>
- [3] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. Sebastopol, CA: O’Reilly Media, 2025.
- [4] M. Kleppmann and C. Riccomini, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 2nd edition. Santa Rosa, CA: O’Reilly Media, 2026.
- [5] C. D. Manning, P. Raghavan, and H. Schütze, “Evaluation in information retrieval,” in *Introduction to Information Retrieval*, Cambridge University Press, pp. 151–175. [Online]. Available: <https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-information-retrieval-1.html>
- [6] S. Bruch, S. Gai, and A. Ingber, “An Analysis of Fusion Functions for Hybrid Retrieval,” *ACM Trans. Inf. Syst.*, vol. 42, no. 1, p. 20:1-20:35, Aug. 2023, doi: 10.1145/3596512.
- [7] X. Liu et al., “A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going?,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 10, pp. 5735–5754, Oct. 2025, doi: 10.1109/TKDE.2025.3592032.
- [8] C. Xu, L. Gao, Y. Miao, and X. Zheng, “Distributed Retrieval-Augmented Generation.” Accessed: Jun. 01, 2026. [Online]. Available: <https://arxiv.org/html/2505.00443v1>
- [9] R. Guerraoui, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, and M. de Vos, “Efficient Federated Search for Retrieval-Augmented Generation,” in *Proceedings of the 5th Workshop on Machine Learning and Systems*, in *EuroMLSys ’25*. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 74–81. doi:

10.1145/3721146.3721942.

[10] X. Yu, P. Jian, and C. Chen, "TableRAG: A Retrieval Augmented Generation Framework for Heterogeneous Document Reasoning," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds., Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 14063–14082. doi: 10.18653/v1/2025.emnlp-main.710.

[11] H. S. Shalaan, T. H. A. Soliman, and A. M. AbdelAziz, "G-SQL: A Schema-Aware and Rule-Guided Approach for Robust Natural Language to SQL Translation," IEEE Access, vol. 13, pp. 158520–158534, 2025, doi: 10.1109/ACCESS.2025.3607879.

[12] N. Yaghmazadeh, Y. Wang, I. Dillig, and T. Dillig, "SQLizer: query synthesis from natural language," Proc. ACM Program. Lang., vol. 1, no. OOPSLA, p. 63:1-63:26, Oct. 2017, doi: 10.1145/3133887.

[13] A. Garba, S. Wu, and S. Khalid, "Federated search techniques: an overview of the trends and state of the art," Knowl Inf Syst, vol. 65, no. 12, pp. 5065–5095, Dec. 2023, doi: 10.1007/s10115-023-01922-6.

[14] J. P. Callan, Z. Lu, and W. B. Croft, "Searching distributed collections with inference networks," in Proceedings of the 18th annual international ACM SIGIR conference on Research and development in information retrieval, in SIGIR '95. New York, NY, USA: Association for Computing Machinery, Jul. 1995, pp. 21–28. doi: 10.1145/215206.215328.

[15] L. Si and J. Callan, "Relevant document distribution estimation method for resource selection," in Proceedings of the 26th annual international ACM SIGIR conference on Research and development in informaion retrieval, in SIGIR '03. New York, NY, USA: Association for Computing Machinery, Jul. 2003, pp. 298–305. doi: 10.1145/860435.860490.

[16] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[17] Q. Chen et al., "SPANN: highly-efficient billion-scale approximate nearest neighbor search," in Proceedings of the 35th International Conference on Neural Information Processing Systems, in NIPS '21. Red Hook, NY, USA: Curran

Associates Inc., Dec. 2021, pp. 5199–5212.

[18] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnawamy, and R. Kadekodi, “DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2019. Accessed: Jun. 25, 2026. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html

[19] “PostgreSQL 18.4 Documentation,” PostgreSQL Documentation. Accessed: Jun. 11, 2026. [Online]. Available: <https://www.postgresql.org/docs/18/index.html>

[20] M. T. Özsu and P. Valduriez, “Distributed and Parallel Database Design,” in *Principles of Distributed Database Systems*, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 33–89. doi: 10.1007/978-3-030-26253-2_2.

[21] J. J. Pan, J. Wang, and G. Li, “Survey of vector database management systems,” *The VLDB Journal*, vol. 33, no. 5, pp. 1591–1615, Jul. 2024, doi: 10.1007/s00778-024-00864-x.

[22] M. T. Özsu and P. Valduriez, “Parallel Database Systems,” in *Principles of Distributed Database Systems*, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 349–394. doi: 10.1007/978-3-030-26253-2_8.

[23] M. T. Özsu and P. Valduriez, “Distributed Query Processing,” in *Principles of Distributed Database Systems*, M. T. Özsu and P. Valduriez, Eds., Cham: Springer International Publishing, 2020, pp. 129–182. doi: 10.1007/978-3-030-26253-2_4.

Appendices

Placeholder